

Project 03 예약 SaaS — Phase 1-1 학습정리

DB 준비: 공유 PostgreSQL에 전용 데이터베이스 + 전용 유저 만들기 (최소 권한 격리)

0. Phase 1-1은 무엇인가

Phase 0에서 "git push → 자동 배포" 파이프라인을 깔았다. 그런데 그때까지 앱은 `/health` 만 응답하는 깃데기라 데이터를 담을 곳(DB)이 없었다. Phase 1(인증 + 멀티테넌시)부터는 사용자·테넌트 데이터를 저장해야 하므로, 그 토대인 데이터베이스와 접속 계정을 먼저 준비한다. 이것이 Phase 1의 첫 단계(1-A)다.

1-1에서 한 일 요약 - 기존 공유 PostgreSQL 서버 `pg-shared-dglee` (PostgreSQL 18)에 `reservationdb` 데이터베이스 생성 - 그 DB에만 권한이 있는 **03 전용 유저** `reservation_app` 생성 (최소 권한) - 권한이 실제로 걸렸는지 **SQL로 검증** - (도중) 무료 평가판 만료 → 종량제(Pay-As-You-Go) 전환 처리

전체 그림 — "그릇은 공유, 내용물은 분리"

```
pg-shared-dglee (PostgreSQL 18 서버) ← 공유 (건물)
├─ tododb ← Todo가 사용 (방 1)
└─ reservationdb ← 03이 사용 (신규) (방 2)
    ▲
    └─ reservation_app 유저: 이 방 열쇠만 보유 (세입자)
        · tododb 등 다른 방은 접근 불가
```

무료 750시간 grant는 서버(건물) 단위로, 방(데이터베이스)을 더 만들어도 추가 컴퓨트 비용이 없다. 그래서 Todo와 한 서버를 공유하되 데이터베이스와 유저로 격리한다.

1. 서버 / 데이터베이스 / 유저 — 개념부터 구분

세 가지가 헷갈리기 쉬운데, 계층이 다르다.

개념	비유	우리 예시
서버(Server)	건물	<code>pg-shared-dglee</code> — PostgreSQL 엔진이 도는 한 대의 인스턴스
데이터베이스(Database)	건물 안의 방	<code>tododb</code> , <code>reservationdb</code> — 서버 안의 독립된 데이터 공간
유저/역할(Role)	세입자(열쇠 보유자)	<code>reservation_app</code> — 특정 방에만 들어갈 수 있는 계정

한 서버 안에 데이터베이스 여러 개, 한 서버 안에 유저(역할) 여러 개가 공존할 수 있다. 유저는 서버 전체에 속하지만, 어떤 데이터베이스/스키마에 접근할 수 있는지는 **권한(GRANT)**으로 따로 정한다.

PostgreSQL의 "역할(Role)" 용어: PostgreSQL은 전통적 의미의 "유저"와 "그룹"을 **역할(role)**이라는 하나의 개념으로 통합했다. `LOGIN` 속성이 있는 역할 = 우리가 아는 "로그인 가능한 유저". 그래서 `CREATE USER` 와 `CREATE ROLE ... WITH LOGIN` 은 사실상 같다.

2. 단계별 상세

STEP 1. PostgreSQL 서버 생존 확인

```
az postgres flexible-server list --query "[].{name:name, rg:resourceGroup, version:version, state:state}" -o table
```

명령어 단어 풀이 - `postgres flexible-server` : Azure 관리형 PostgreSQL(Flexible Server 종류)을 다루는 명령 그룹. - `list` : 내 구독의 PostgreSQL 서버 목록. - `--query "[...].{name:name, ...}"` : 결과에서 원하는 필드만 골라 표시(JMESPath 질의). - `-o table` : 출력을 표로. 결과: `pg-shared-dglee / rg-deploy-test / 버전 18` 확인. PostgreSQL 18은 최신이라 `xmin` 낙관적 잠금 등 우리가 쓸 기능이 모두 지원되고, EF Core 10 + Npgsql 10과도 호환된다.

겪은 일: `state:state` 를 `state:sate` 로 오타내 State 컬럼이 비어 나왔다. 동작엔 무관(나머지 필드는 정상)하지만, 오타가 결과를 조용히 누락시킬 수 있다는 점을 기억.

STEP 2. 데이터베이스 생성

```
az postgres flexible-server db create --resource-group rg-deploy-test --server-name pg-shared-dglee --database-name reservationdb
```

명령어 단어 풀이 - `db create` : 서버 **안에** 데이터베이스를 만든다. (서버를 만드는 게 아니라, 이미 있는 서버에 방을 하나 낸다.) - `--resource-group rg-deploy-test` : 서버가 속한 리소스 그룹. - `--server-name pg-shared-dglee` : 어느 서버 안에 만들지. - `--database-name reservationdb` : 만들 데이터베이스 이름(소문자로 통일).

deprecated 경고: `--database-name` 이 향후 `--name` 으로 바뀐다는 경고가 떴으나, 현재 버전(2.86)에선 그대로 작동.

STEP 3. (사건) 무료 평가판 만료 → 종량제 전환

DB 생성을 시도하자 다음 에러가 발생했다.

```
(ReadOnlyDisabledSubscription) The subscription is disabled and therefore marked as read only. You cannot perform any write actions until it is re-enabled.
```

원인: Azure 무료 평가판(30일 + \$200 크레딧)이 만료되어, 구독이 **읽기 전용**으로 잠김. 조회(read)는 되지만 생성·수정(write)은 모두 거부된다. (요금 폭주를 막는 Azure의 안전장치이기도 하다.)

해결: Azure 포털 → 구독 → **종량제(Pay-As-You-Go)로 업그레이드** + 결제 수단 등록. - 지원 계획은 **"기본 - 포함됨"(무료)** 선택. 개발자(\$29)·표준(\$100)·전문가(\$1,000)는 **기술 지원**을 사는 유료 옵션이며 서비스 사용료와 무관 → 선택하지 않음. - 종량제 = "무료 한도(매월 85개 이상 서비스)를 초과한 사용량만 카드로 청구"하는 방식.

용어 — 종량제(Pay-As-You-Go): 사전 약정 없이, 실제 사용한 만큼만 후불로 청구되는 요금제. 무료 평가판이 끝난 개인이 계속 쓰려면 이 방식으로 전환한다.

STEP 4. (필수) 비용 안전장치 — 예산 알림

종량제 전환 직후, 실제 과금이 시작되므로 **예산 알림**을 먼저 설정. - 포털 → 비용 관리 → 예산(Budgets) → 추가 → 월별 ₩30,000, 임계값 50/80/100%에서 이메일 알림.

예산 알림은 과금을 막지 않는다. "이만큼 썼다"고 메일로 알리는 **조기경보**일 뿐. 진짜 비용 방어는 ① Container Apps scale-to-zero, ② 안 쓸 때 PostgreSQL 정지(`az postgres flexible-server stop`), ③ 안 쓰는 리소스 삭제.

STEP 5. 대소문자 합정 — `reservationDB` → `reservationdb`

처음 만들 때 `reservationDB` (대문자 포함)로 생성했다가, 빈 DB일 때 삭제 후 `reservationdb` (소문자)로 재생성했다.

```
az postgres flexible-server db delete --resource-group rg-deploy-test --server-name pg-shared-dglee --database-name reservationDB
az postgres flexible-server db create --resource-group rg-deploy-test --server-name pg-shared-dglee --database-name reservationdb
```

왜 소문자인가: PostgreSQL은 따옴표 없이 쓴 식별자를 자동으로 소문자로 처리한다. 대문자가 섞인 이름은 이후 연결 문자열·쿼리에서 대소문자를 정확히 맞춰야만 인식되어 실수의 원인이 된다. **DB·테이블 이름은 소문자로 통일**하는 것이 관례. 빈 DB일 때 바로잡는 것이 비용이 가장 싸다(데이터가 들어간 뒤엔 변경 불가).

STEP 6. 전용 유저 생성 + 권한 부여 (SQL)

DB 유저 생성은 Azure CLI가 아니라 **PostgreSQL 내부의 일**이라, DB 클라이언트(VS Code의 PostgreSQL 확장)로 서버에 **관리자로 접속**해 SQL을 실행한다.

```
-- ① 03 전용 유저(역할) 생성 [어느 DB에서 실행해도 됨]
CREATE ROLE reservation_app WITH LOGIN PASSWORD '****';

-- ② reservationdb 접속 권한 부여 [어느 DB에서 실행해도 됨]
GRANT CONNECT ON DATABASE reservationdb TO reservation_app;

-- ③ public 스키마 사용/생성 권한 [반드시 reservationdb에 접속해서 실행]
GRANT USAGE, CREATE ON SCHEMA public TO reservation_app;

-- ④ 앞으로 만들 테이블에 대한 기본 권한 [반드시 reservationdb에 접속해서 실행]
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO reservation_app;
```

SQL 한 줄씩 풀이 - **CREATE ROLE reservation_app WITH LOGIN PASSWORD '...'**: 로그인 가능한 역할(=유저)을 생성. **WITH LOGIN** 이 있어야 이 계정으로 접속 가능. **PASSWORD** 는 이 계정 전용 암호(관리자 암호와 별개). - **GRANT CONNECT ON DATABASE reservationdb TO reservation_app**: 이 유저에게 **reservationdb** 에 **접속할 권한**을 부여. = "방 열쇠 지급". tododb엔 부여하지 않으므로 접근 불가. - **GRANT USAGE, CREATE ON SCHEMA public**: **reservationdb** 안의 **public** 스키마(작업 공간)에서 **사용(USAGE)·테이블 생성(CREATE)** 권한. EF Core 마이그레이션이 테이블을 만들려면 CREATE가 필수. - **ALTER DEFAULT PRIVILEGES ... GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES**: "앞으로 이 스키마에 만들어질 테이블들에 대해 **조회·삽입·수정·삭제를 기본 허용**". 이게 없으면 테이블은 만들어도 데이터 조작 권한이 없어 곤란해진다.

용어 — 스키마(Schema): 데이터베이스 안에서 테이블·뷰 등을 묶는 **네임스페이스(작업 공간)**. PostgreSQL은 기본적으로 **public** 스키마를 제공하며, 별도 설정이 없으면 테이블은 여기에 만들어진다.

②는 어디서 실행해도 되고 ③④는 왜 reservationdb에서 해야 하나: ②(접속 권한)는 "데이터베이스 객체 자체"에 거는 권한이라 어느 연결에서 실행하든 대상이 명확하다. 반면 ③④(스키마·테이블 권한)는 **현재 접속한 데이터베이스의 내부 공간**을 대상으로 하므로, 반드시 **reservationdb** 에 접속한 상태에서 실행해야 그 DB의 public 스키마에 정확히 걸린다. (방 밖에서 "이 방 가구 써도 돼"라고 지정할 수 없는 것과 같다.)

STEP 7. 권한 적용 검증 (★ 학습 포인트)

권한이 실제로 걸렸는지 SQL로 확인했다.

```
-- 검증 1: 유저 존재 + 로그인 권한 [아무 DB]
SELECT rolname, rolcanlogin FROM pg_roles WHERE rolname = 'reservation_app';

-- 검증 2: reservationdb 접속 권한 [아무 DB]
SELECT has_database_privilege('reservation_app', 'reservationdb', 'CONNECT') AS can_connect;

-- 검증 3: public 스키마 권한 [반드시 reservationdb에 접속]
SELECT
  has_schema_privilege('reservation_app', 'public', 'USAGE') AS usage_priv,
  has_schema_privilege('reservation_app', 'public', 'CREATE') AS create_priv;
```

결과: 모두 **true** → 권한 정상 적용 확인.

검증 함수 풀이 - **pg_roles**: PostgreSQL이 제공하는 시스템 카탈로그(역할 목록 뷰). **rolcanlogin** 이 로그인 가능 여부. -

has_database_privilege(유저, DB, 권한): 해당 유저가 그 DB에 그 권한을 갖는지 true/false 반환. - **has_schema_privilege(유저, 스키마, 권한)**: 해당 유저가 그 스키마에 그 권한을 갖는지 true/false 반환. 검증 3은 반드시 reservationdb에 접속해 실행해야 그 DB 기준으로 답한다.

3. 최소 권한 원칙 (이 단계의 핵심 사상)

전용 유저를 따로 만든 이유는 **최소 권한 원칙(Principle of Least Privilege)** 이다.

- 서버 관리자 계정 = **마스터키**. 모든 DB 접근·생성·삭제 가능. 03 앱이 이 키를 들고 있다가 연결 문자열이 유출되면 **Todo 데이터까지 위험**.
- reservation_app** = **reservationdb** 방 열쇠만 보유. 유출돼도 피해가 reservationdb 안으로 한정. Todo는 안전.

이 사고방식은 프로젝트 전반에서 일관된다: - **Phase 0 OIDC**: 배포 신분증의 권한을 `--scope` 로 RG 하나에 가둠. - **Phase 1-1 (지금)**: DB 유저 권한을 `reservationdb` 하나에 가둠. - **03 멀티테넌시(예정)**: 테넌트별 데이터 격리. - **05 DB모니터링(예정)**: 읽기 전용 유저. "필요한 곳에, 필요한 만큼만" — 같은 원칙의 서로 다른 적용.

4. 겪은 에러·이슈와 교훈

이슈	원인	교훈
<code>ReadOnlyDisabledSubscription</code>	무료 평가판 만료로 구독이 읽기 전용	무료 계정은 30일/\$200 소진 시 잠긴다. 종량제 전환 필요
<code>state:sate</code> 오타로 컬럼 누락	쿼리 필드명 오타	오타는 에러 없이 결과를 조용히 누락시킬 수 있다
DB 이름 대문자(<code>reservationDB</code>)	소문자 규칙 미준수	식별자는 소문자 통일. 빈 상태에서 바로잡기
암호 채팅 노출	부주의	암호는 본인 환경에서만 입력. 노출 시 <code>ALTER ROLE ... PASSWORD</code> 로 교체
③④ 권한이 엉뚱한 DB에 걸릴 위험	스키마 권한을 다른 DB에서 실행	스키마/테이블 권한은 대상 DB에 접속해 실행

미완 숙제: 채팅에 노출된 `reservation_app` 암호는 연결 문자열에 박히기 전(1-E)에 `ALTER ROLE reservation_app PASSWORD '새암호';` 로 반드시 교체한다.

5. 비용 메모 (종량제 전환 이후)

항목	예상 비용	절감법
Container Apps (app-todo, app-reservation)	거의 \$0	scale-to-zero (유휴 시 과금 없음)
관리형 PostgreSQL <code>pg-shared-dglee</code>	최약 월 \$12~15	안 쓸 때 <code>flexible-server stop</code> (최대 7일)
ACR <code>acrdglee0412</code> (Basic)	~\$5/월	상시 무료 없음. 안 쓰면 삭제

FinOps 관점(면접 카드): 무료에서 종량제로 전환되며 "비용을 의식하는 운영"이 필요해졌다. 개발·데모 외 시간엔 DB를 정지하고, scale-to-zero로 유휴 비용을 없애는 식의 비용 거버넌스를 적용. "무료라 신경 안 쓴" 것보다 "실제 과금 환경에서 비용을 통제함" 경험이 더 가치 있다.

6. 면접에서 말할 수 있는 포인트

- 공유 인프라 + 논리적 격리** — 한 PostgreSQL 서버를 데이터베이스·유저 단위로 나눠 멀티 프로젝트를 공존시켰다. 비용(무료 grant는 서버 단위)과 격리를 동시에 달성.
- 최소 권한 원칙의 일관 적용** — DB 유저 권한을 `reservationdb` 하나로 제한. OIDC의 `--scope` 제한, 멀티테넌시 격리와 같은 사고방식.
- 서버/DB/유저 계층 이해** — 무엇이 서버 전역이고 무엇이 DB 로컬인지(②는 DB 객체 권한, ③④는 스키마 권한) 구분해 SQL을 적절한 위치에서 실행.
- 권한을 검증하는 습관** — `has_*_privilege` 함수로 의도한 권한이 실제로 걸렸는지 확인. "걸었다"와 "걸렸다"는 다르다.
- 비용 거버넌스** — 종량제 전환 후 예산 알림·DB 정지·scale-to-zero로 비용 통제.

7. 다음 — Phase 1-2 (클린 아키텍처 골격)

DB 그릇이 준비됐으니, 다음은 **솔루션을 클린 아키텍처 4계층으로 분리**한다. - Domain / Application / Infrastructure / Presentation 프로젝트 분리. - 단, 실용적으로: 핵심 로직(멀티테넌시·낙관적 잠금)만 제대로 계층을 타고 단순 CRUD는 가볍게. - 처음부터 골격을 세우는 이유: 나중에 분리하는 비용 > 처음에 칸 나누는 비용. Dockerfile 빌드 경로도 이때 솔루션 구조에 맞춰 한 번 정리. - 이후 1-C(엔티티) → 1-D(EF Core 연결·첫 마이그레이션·자동 적용) → 1-E(JWT, Access/Refresh, httpOnly 쿠키, Token Rotation) → 1-F(Global Query Filter 멀티테넌시) → 1-G(배포 검증)로 진행.

Phase 1-1 학습정리 끝. 작성 기준: PostgreSQL 18 / Azure CLI 2.86 / VS Code PostgreSQL 확장.